microMC Nuclear Data Structures

A rigorous guide using a small but faithful toy library

Purpose. This document explains, in concrete terms, how nuclear data is stored in the `microMC` style data layout: what each data structure means, how the global pools are organized, how offsets reconstruct ownership, why `threshold_idx` exists, and how secondary distributions use incident-energy grids that are distinct from the parent cross-section grid.

Scope. The guide uses a compact toy library containing four nuclides - H1, O16, U235, and U238 - with only physically sensible reaction channels included for each. The numerical values are illustrative, not evaluated data. The goal is to teach the storage model faithfully.

Compiled from LaTeX for reference and annotation

Contents

1 How to read this document **2**

2 The central idea of the storage model **2**
2.1 Why this is useful . . . 2
2.2 The two struct tiers: owner and view . . . 3

3 The three different kinds of energy grid **3**
3.1 Parent nuclide cross-section grid . . . 3
3.2 Reaction support on the parent grid . . . 3
3.3 Secondary-distribution incident-energy grids . . . 3

4 The toy library used throughout **3**
4.1 Parent grids . . . 4
4.2 Reaction inventory . . . 4

5 Direct explanation of each core data structure **4**
5.1 `NuclideDescriptor` . . . 4
5.2 `ReactionDescriptor` . . . 5
5.2.1 What `dist_id` points to, by reaction type . . . 6
5.3 `AngularDistPool` — the two-level ragged array . . . 6
5.3.1 Concrete example with offsets . . . 6
5.4 `EnergyDistPool` . . . 7
5.5 `KalbachMannDistPool` . . . 7
5.6 `FissionYieldDescriptor` and `FissionYieldPool` . . . 7
5.7 `DelayedNeutronGroupDescriptor` and `DelayedFissionDescriptor` . . . 8
5.8 `Material` . . . 8

6 The toy reaction supports **8**
6.1 H1 . . . 9
6.2 O16 . . . 9
6.3 U235 . . . 9
6.4 U238 . . . 9

7 The global `reactions[]` array in the toy library **9**

8 Direct explanation of the global pools **10**
8.1 The global `xs_pool` . . . 10
8.2 The global angular pool . . . 11
8.3 The Kalbach/continuum pool . . . 11
8.4 The fission outgoing-energy pool . . . 11
8.5 The fission-yield pool . . . 11

9 `GridIndex` and the binary search **12**

10 Stochastic energy interpolation **12**

11 Worked interpretation examples **13**
11.1 Example 1: O16 continuum inelastic . . . 13
11.2 Example 2: U235 fission . . . 13

12 Full collision pipeline **14**
12.1 Step 1: Macroscopic cross section and reaction sampling . . . 14
12.2 Step 2: Distance to collision . . . 14
12.3 Step 3: Target velocity sampling . . . 14
12.4 Step 4: Dispatch to collision handler . . . 14
12.5 Pipeline A: Elastic scatter . . . 14
12.6 Pipeline B: Discrete inelastic scatter . . . 15
12.7 Pipeline C: Continuum inelastic scatter . . . 15
12.8 Pipeline D: Fission . . . 15
12.9 Pipeline E: Absorption . . . 16
12.10 Pipeline F: (n,2n) and (n,3n) . . . 16

13 How a lookup proceeds in practice **16**

14 What the design gets right **17**

15 Common conceptual mistakes to avoid **17**

16 Final mental model **18**

1 How to read this document

This guide is intentionally concrete. It does not start from abstract software architecture. It starts from the actual questions that matter when reading or writing the code:

- What is owned directly, and what is only referenced indirectly?
- What does each field in each structure actually mean?
- Why are there offsets everywhere?
- What is the difference between a parent nuclide energy grid and a secondary-distribution incident-energy grid?
- When a neutron interacts with a nuclide, how do the structures get traversed in practice?
- How does the code sample a reaction and process the collision?

The toy library is small enough to inspect by eye, but it keeps the important structural features:

- each nuclide has its own parent cross-section grid,
- reactions only use slices of that parent grid,
- secondary distributions have independent incident-energy grids,
- fission needs more than one auxiliary pool (prompt spectrum, $\bar{\nu}$, delayed groups),
- aggregate channels such as non-fission absorption require care.

2 The central idea of the storage model

The design avoids nested ownership. In a naive object model you might expect the following:

```
Nuclide
  owns energy grid
  owns reactions
    each reaction owns xs array
    each reaction owns angular law
    each reaction owns outgoing-energy law
```

That is *not* how this layout works. Instead, the design uses:

1. a few large global arrays (energy grids, cross sections, secondary tables), and
2. small descriptor structures that say where each relevant slice begins and how long it is.

So a reaction does not "contain" its cross section table. A reaction descriptor says, in effect, "my cross section values begin at offset 148 in the global `xs_pool`, my table has length 12, and the first value corresponds to parent-grid index 4."

2.1 Why this is useful

This gives several practical benefits:

- compact storage,
- no deep heap allocation tree,
- easy serialization to a binary file,

- easier transfer to accelerators (the `NuclearData` view struct is pure POD),
- one consistent lookup strategy based on offsets.

The price is that the code can look alien at first. It is much more like an indexing system than like classical object-oriented encapsulation.

2.2 The two struct tiers: owner and view

`NuclearDataHost` owns the heap via `std::vector` members. It is never passed to physics routines. Instead, `host.view()` returns a `NuclearData` struct whose fields are raw pointers into those vectors. `NuclearData` is pure POD — no `std::` types — so it can be `memcpy`'d to GPU device memory unchanged. All physics code takes `const NuclearData&`.

3 The three different kinds of energy grid

A major source of confusion is the word "energy grid." There is not just one of them.

3.1 Parent nuclide cross-section grid

Each nuclide has its own parent cross-section grid. This is the grid stored in the global `energy_grids` pool and referenced by a `NuclideDescriptor`. Reaction cross sections are aligned against this parent grid using `threshold_idx`.

3.2 Reaction support on the parent grid

A reaction usually does not use the whole parent grid. Instead it uses a suffix or other contiguous slice of it. This support is not stored as a separate energy array. It is implied by three things:

- the parent grid,
- `threshold_idx`,
- `xs_length`.

If the parent grid has 9 points and a reaction has `threshold_idx = 4`, `xs_length = 5`, then the reaction occupies parent local indices 4 through 8.

3.3 Secondary-distribution incident-energy grids

Angular laws, continuum/Kalbach distributions, fission outgoing-energy distributions, and $\bar{\nu}(E)$ laws all have their own incident-energy grids. These should not be assumed equal to the parent grid. In practice they are often much coarser and placed only where the secondary law changes enough to justify tabulation.

Parent grid
Owned by nuclide descriptor. Used to interpret reaction cross-section support.

Reaction support
Implicit slice of the parent grid, reconstructed using `threshold_idx` and `xs_length`.

Secondary grid
Stored in a separate pool. Used for angular laws, Kalbach laws, prompt fission spectra, or $\bar{\nu}(E)$.

4 The toy library used throughout

The toy library contains four nuclides with intentionally different parent grids.

4.1 Parent grids

Nuclide	Parent cross-section grid
H1	[1e-5, 1e-3, 1e-1, 1e0, 1e2, 1e4, 1e6]
O16	[1e-3, 1e-1, 1e0, 1e1, 1e3, 1e5, 1e6, 1e7]
U235	[1e-5, 1e-3, 1e-1, 1e0, 1e1, 1e3, 1e4, 1e5, 1e6]
U238	[1e-5, 1e-2, 1e0, 1e2, 1e3, 1e5, 1e6, 1e7, 2e7]

The global `energy_grids` pool is just the concatenation of these four arrays.

```
energy_grids = [
  // H1: indices 0..6
  1e-5, 1e-3, 1e-1, 1e0, 1e2, 1e4, 1e6,
  // O16: indices 7..14
  1e-3, 1e-1, 1e0, 1e1, 1e3, 1e5, 1e6, 1e7,
  // U235: indices 15..23
  1e-5, 1e-3, 1e-1, 1e0, 1e1, 1e3, 1e4, 1e5, 1e6,
  // U238: indices 24..32
  1e-5, 1e-2, 1e0, 1e2, 1e3, 1e5, 1e6, 1e7, 2e7
]
```

4.2 Reaction inventory

Only channels that are physically meaningful for each nuclide are included. Note that in the actual `microMC` codebase, non-fission absorption (MT=102–117) is stored as a single summed `ABSORPTION` channel. There is no separate "capture" reaction — capture is folded into absorption. The toy library follows this convention:

Nuclide	Included reactions
H1	elastic, absorption
O16	elastic, discrete inelastic, continuum inelastic, absorption, n2n, n3n
U235	elastic, discrete inelastic, continuum inelastic, fission, absorption, n2n, n3n
U238	elastic, discrete inelastic, continuum inelastic, fission, absorption, n2n, n3n

5 Direct explanation of each core data structure

This section explains each structure directly, field by field, in plain language.

5.1 `NuclideDescriptor`

This structure answers two questions:

- where does this nuclide's parent cross-section grid live?
- where does this nuclide's reaction block live?

Field	Meaning

`grid_offset`	Starting index of this nuclide's parent energy grid inside the global `energy_grids` pool.
`grid_length`	Number of energy points in the parent grid.
`A`	Atomic mass ratio (target mass / neutron mass). Used for kinematics.
`rxn_offset`	Starting index of this nuclide's reaction block in the global `reactions` array.
`n_reactions`	Number of reactions owned by this nuclide.
`zaid`	ZAID identifier (e.g. 1001 for H1, 92238 for U238).

For the toy library:

```
nuclides[0] = { grid_offset=0, grid_length=7, A=1.0, rxn_offset=0, n_reactions=2, zaid
    =1001 }
nuclides[1] = { grid_offset=7, grid_length=8, A=16.0, rxn_offset=2, n_reactions=6, zaid
    =8016 }
nuclides[2] = { grid_offset=15, grid_length=9, A=235.0, rxn_offset=8, n_reactions=7, zaid
    =92235 }
nuclides[3] = { grid_offset=24, grid_length=9, A=238.0, rxn_offset=15, n_reactions=7, zaid
    =92238 }
```

This means, for example, that U235 owns:

- `energy_grids[15..23]`, and
- `reactions[8..14]`.

5.2 `ReactionDescriptor`

This structure answers three questions:

- where do this reaction's cross-section values live?
- where do they begin on the parent grid?
- where is the reaction's outgoing-data metadata stored?

Field	Meaning
`type`	Reaction category. One of: `ELASTIC`, `DISCRETE_INELASTIC`, `CONTINUUM_INELASTIC`, `FISSION`, `ABSORPTION`, `N2N`, `N3N`.
`xs_offset`	Starting index in the global `xs_pool` where this reaction's tabulated cross-section values begin.
`xs_length`	Number of tabulated cross-section values for this reaction.
`threshold_idx`	Parent-grid local index corresponding to the first cross-section entry. This is the key field that reconstructs reaction support on the parent grid.
`dist_id`	Index into the relevant outgoing-distribution pool. Which pool depends on reaction type (see below). Set to -1 for absorption (no secondaries).
`Q_value`	Reaction Q-value in eV. Only physically meaningful for discrete inelastic.
`multiplicity`	Number of outgoing neutrons. 1 for elastic/inelastic, 2 for (n,2n), 3 for (n,3n), -1 for fission (energy-dependent $\bar{\nu}$).

`yield_id`	Index into the `FissionYieldPool`. Only meaningful for fission (-1 otherwise).
`delayed_id`	Index into `delayed_fission_descs`. Only meaningful for fission (-1 otherwise).

5.2.1 What `dist_id` points to, by reaction type

The `dist_id` field is polymorphic. Its target pool depends on the reaction type:

Reaction type	Pool indexed by `dist_id`	What is sampled
`ELASTIC`	`data.angular`	scattering cosine μ_{cm}
`DISCRETE_INELASTIC`	`data.angular`	scattering cosine μ_{cm}
`CONTINUUM_INELASTIC`	`data.kalbach`	E'_{out} and μ
`FISSION`	`data.fission_energy`	outgoing energy E'
`N2N`, `N3N`	`data.kalbach`	E'_{out} and μ
`ABSORPTION`	— (unused)	nothing (`dist_id` $= -1$)

The `switch` in the transport loop dispatches to the correct collision handler, which knows which pool to pass `dist_id` into. For example, `elastic_scatter` calls `sample_cosine(data.angular, rxn.dist_id, ...)`, while `inelastic_scatter_cont` calls `sample_kalbach_mann(data.kalbach, rxn.dist_id, ...)`.

5.3 `AngularDistPool` — the two-level ragged array

This pool stores angular laws for elastic and discrete inelastic reactions. Logically, one angular distribution looks like this:

```
Distribution k:
  incident energy E1 -> mu/pdf/cdf table T1 (n1 bins)
  incident energy E2 -> mu/pdf/cdf table T2 (n2 bins)
  incident energy E3 -> mu/pdf/cdf table T3 (n3 bins)
```

The pool flattens this ragged structure into global arrays using two levels of indirection:

- **Level 1:** `energy_offsets[k]` to `energy_offsets[k+1]` gives the range of incident-energy entries belonging to distribution `k`. The actual energy values are in the `energies` array at those positions.
- **Level 2:** For each incident-energy entry `j`, `table_offsets[j]` to `table_offsets[j+1]` gives the range of μ/pdf/cdf bins belonging to that entry.

5.3.1 Concrete example with offsets

Suppose distribution 0 (H1 elastic) has 3 incident energies, each with a different number of μ bins, and distribution 1 (O16 elastic) has 2 incident energies:

```
Dist 0 (H1 elastic):
  E=1e-3 -> 5 mu bins
  E=1e2 -> 5 mu bins
  E=1e6 -> 8 mu bins

Dist 1 (O16 elastic):
  E=1e-1 -> 5 mu bins
  E=1e5 -> 6 mu bins
```

The flattened arrays would be:

```
energy_offsets = [0, 3, 5, ...]
                 | | |
                 | | +-- dist 2 starts at energy entry 5
                 | +-- dist 1 starts at energy entry 3
                 +-- dist 0 starts at energy entry 0

energies = [1e-3, 1e2, 1e6, 1e-1, 1e5, ...]
           |---- dist 0 ---| |- dist 1 -|

table_offsets = [0, 5, 10, 18, 23, 29, ...]
                | | | | | |
                | | | | | +-- end of dist1/E=1e5 (6 bins)
                | | | | +-- start of dist1/E=1e5
                | | | +-- end of dist0/E=1e6 (8 bins)
                | | +-- start of dist0/E=1e6
                | +-- start of dist0/E=1e2 (5 bins)
                +-- start of dist0/E=1e-3 (5 bins)

mu = [mu values for all tables, concatenated]
pdf = [pdf values for all tables, concatenated]
cdf = [cdf values for all tables, concatenated]
```

To sample from distribution k at incident energy E:

1. Use `energy_offsets[k]` and `energy_offsets[k+1]` to find the range of incident energies.
2. Use stochastic energy interpolation (section 10) to pick one incident-energy entry j.
3. Use `table_offsets[j]` and `table_offsets[j+1]` to find the μ/cdf slice.
4. Sample from the CDF to get the scattering cosine.

5.4 EnergyDistPool

This pool stores outgoing-energy laws tabulated at incident energies. It is used for prompt fission spectra and delayed fission spectra. The structure is identical to the angular pool:

- `energy_offsets` → `energies`: which incident energies belong to each distribution,
- `table_offsets` → `E_out/pdf/cdf`: the outgoing-energy tables for each incident energy.

The same two-level ragged array pattern, the same access logic. Only the leaf data differs (E'_{out} instead of μ).

5.5 KalbachMannDistPool

This pool is a specialised version of an outgoing-energy distribution pool. It has the same two-level structure as `EnergyDistPool`, but the leaf tables store two extra arrays: the Kalbach-Mann parameters `r` and `a`. These are stored parallel to `E_out/pdf/cdf`, indexed by the same `table_offsets`. Channels like continuum inelastic, (n,2n), and (n,3n) use this pool.

5.6 FissionYieldDescriptor and FissionYieldPool

This pool stores tabulated $\bar{\nu}(E)$ laws. One entry is logically just a pair of arrays:

```
energy = [E1, E2, E3, ...]
nu_bar = [v1, v2, v3, ...]
```

The descriptor tells you where a particular law begins in the global `fy_energy` and `fy_nu_bar` arrays and how many points it has. This is a single-level indirection — simpler than the angular/energy pools because $\bar{\nu}$ is a scalar function of E, not a distribution.

5.7 `DelayedNeutronGroupDescriptor` and `DelayedFissionDescriptor`

These structures let a fission reaction reference delayed neutron metadata compactly. The chain is:

1. The reaction's `delayed_id` points to a `DelayedFissionDescriptor`.
2. The `DelayedFissionDescriptor` has `group_offset` and `n_groups`, giving a slice of the `delayed_groups` array.
3. Each `DelayedNeutronGroupDescriptor` in that slice has:
   - `yield_id` — index into `FissionYieldPool` for this group's $\bar{\nu}_d(E)$,
   - `dist_id` — index into `data.delayed_fission_energy` (an `EnergyDistPool`) for this group's delayed spectrum.

5.8 `Material`

A material binds nuclides together with number densities and a temperature. It is not part of the nuclear data — it is a user-defined composition.

Field	Meaning
`name`	Human-readable label.
`density`	Input density. Negative = mass density (g/cm^3), positive = atomic density (atoms/barn-cm), zero = direct number densities.
`temperature`	Material temperature in Kelvin. Used for target motion sampling.
`zaids[]`	Array of ZAID identifiers for each nuclide in the mix.
`number_quantity[]`	Input atom or mass fractions (sign determines interpretation).
`n_nuclides`	Number of active nuclides.
`number_densities[]`	Computed by `resolve_material`: absolute number densities N_i in atoms/barn-cm.
`nuclide_ids[]`	Computed by `resolve_material`: index into `NuclearData::nuclides[]` for each ZAID.

`resolve_material(mat, data)` does two things: (1) maps each ZAID to its index in the nuclear data, and (2) converts the input density and fractions into absolute number densities using atomic masses from the ENDF/B-VII.1 table. All subsequent physics uses `number_densities[]` and `nuclide_ids[]`.

6 The toy reaction supports

This section records where each reaction begins on its parent grid. This is what `threshold_idx` and `xs_length` encode.

6.1 H1

Parent grid length: 7

Reaction	thr idx	xs len	Parent local support
elastic	0	7	0..6
absorption	2	5	2..6

6.2 O16

Parent grid length: 8

Reaction	thr idx	xs len	Parent local support
elastic	0	8	0..7
discrete inelastic	3	5	3..7
continuum inelastic	4	4	4..7
absorption	2	6	2..7
n2n	4	4	4..7
n3n	5	3	5..7

6.3 U235

Parent grid length: 9

Reaction	thr idx	xs len	Parent local support
elastic	0	9	0..8
discrete inelastic	3	6	3..8
continuum inelastic	4	5	4..8
fission	2	7	2..8
absorption	2	7	2..8
n2n	4	5	4..8
n3n	5	4	5..8

6.4 U238

Parent grid length: 9

Reaction	thr idx	xs len	Parent local support
elastic	0	9	0..8
discrete inelastic	3	6	3..8
continuum inelastic	4	5	4..8
fission	3	6	3..8
absorption	2	7	2..8
n2n	4	5	4..8
n3n	5	4	5..8

7 The global `reactions[]` array in the toy library

The reactions are laid out nuclide by nuclide.

Idx	Reaction	xs offset	xs len	thr idx	Notes
0	H1 elastic	0	7	0	angular dist 0
1	H1 absorption	7	5	2	dist_id = −1
2	O16 elastic	12	8	0	angular dist 1
3	O16 disc inel	20	5	3	angular dist 2
4	O16 cont inel	25	4	4	Kalbach dist 0
5	O16 absorption	29	6	2	dist_id = −1
6	O16 n2n	35	4	4	Kalbach dist 1
7	O16 n3n	39	3	5	Kalbach dist 2
8	U235 elastic	42	9	0	angular dist 3
9	U235 disc inel	51	6	3	angular dist 4
10	U235 cont inel	57	5	4	Kalbach dist 3
11	U235 fission	62	7	2	fission dist 0, yield 0, delayed 0
12	U235 absorption	69	7	2	dist_id = −1
13	U235 n2n	76	5	4	Kalbach dist 4
14	U235 n3n	81	4	5	Kalbach dist 5
15	U238 elastic	85	9	0	angular dist 5
16	U238 disc inel	94	6	3	angular dist 6
17	U238 cont inel	100	5	4	Kalbach dist 6
18	U238 fission	105	6	3	fission dist 1, yield 1, delayed 1
19	U238 absorption	111	7	2	dist_id = −1
20	U238 n2n	118	5	4	Kalbach dist 7
21	U238 n3n	123	4	5	Kalbach dist 8

The exact xs values are not the point here. The important point is that `xs_offset`, `xs_length`, and `threshold_idx` together tell you exactly how a reaction is embedded in the global pools.

8 Direct explanation of the global pools

8.1 The global `xs_pool`

The `xs_pool` is one long array containing every reaction's xs values. The ordering above means:

- H1 elastic occupies `xs_pool[0..6]`,
- H1 absorption occupies `xs_pool[7..11]`,
- O16 elastic occupies `xs_pool[12..19]`,
- U235 fission occupies `xs_pool[62..68]`,
- U238 n3n occupies `xs_pool[123..126]`,
- and so on.

The pool is just a warehouse shelf. Every reaction descriptor stores the shelf location for its own xs slice.

8.2 The global angular pool

The angular pool stores angular laws for elastic and discrete inelastic reactions. For the toy library, suppose the incident-energy grids are:

Distribution	Incident-energy grid
angular 0 (H1 elastic)	[1e-3, 1e2, 1e6]
angular 1 (O16 elastic)	[1e-1, 1e1, 1e5]
angular 2 (O16 disc inel)	[1e1, 1e5, 1e7]
angular 3 (U235 elastic)	[1e-3, 1e1, 1e4]
angular 4 (U235 disc inel)	[1e0, 1e3, 1e6]
angular 5 (U238 elastic)	[1e-2, 1e2, 1e6]
angular 6 (U238 disc inel)	[1e2, 1e5, 2e7]

These incident-energy points are flattened into one array, and `energy_offsets` says where each distribution begins. Then `table_offsets` says where each `mu/pdf/cdf` table begins. See section 5.3 for the concrete two-level indirection walkthrough.

8.3 The Kalbach/continuum pool

For the toy library, suppose the continuum-style incident-energy grids are:

Distribution	Incident-energy grid
Kalbach 0 (O16 cont inel)	[1e3, 1e5, 1e7]
Kalbach 1 (O16 n2n)	[5e3, 1e6]
Kalbach 2 (O16 n3n)	[1e5, 1e7]
Kalbach 3 (U235 cont inel)	[1e1, 1e4, 1e6]
Kalbach 4 (U235 n2n)	[1e3, 1e5]
Kalbach 5 (U235 n3n)	[1e4, 1e6]
Kalbach 6 (U238 cont inel)	[1e2, 1e5, 2e7]
Kalbach 7 (U238 n2n)	[1e3, 1e6]
Kalbach 8 (U238 n3n)	[1e5, 2e7]

Again, these are independent from the parent xs grids.

8.4 The fission outgoing-energy pool

Two prompt fission spectrum distributions:

- fission dist 0 for U235 with incident energies [1e-1, 1e1, 1e5],
- fission dist 1 for U238 with incident energies [1e2, 1e5, 2e7].

Each incident energy has an outgoing-energy pdf/cdf table.

8.5 The fission-yield pool

Two $\bar{\nu}(E)$ laws:

- yield 0 for U235 with incident energies [1e-1, 1e2, 1e6],
- yield 1 for U238 with incident energies [1e2, 1e5, 2e7].

Each law is stored as a slice of the global `fy_energy` and `fy_nu_bar` arrays.

9 GridIndex and the binary search

When the code needs cross sections for a nuclide at energy E, it does not binary-search the energy grid once per reaction. Instead, it searches *once* and reuses the result across all reactions of that nuclide. The result is stored in a `GridIndex`:

```
struct GridIndex {
  int bin; // index of the grid interval containing E
  double frac; // interpolation fraction within [E_lo, E_hi]
};
```

`find_grid_index(data, nuc, E)` does:

1. Binary search on `energy_grids[nuc.grid_offset .. nuc.grid_offset + nuc.grid_length - 1]` to find the interval [`E_lo`, `E_hi`] containing E.
2. Returns `bin` (the index of E_{lo} relative to the nuclide's grid start) and `frac` $= (E - E_{\text{lo}})/(E_{\text{hi}} - E_{\text{lo}})$.

Then `lookup_micro_xs_at(data, rxn, gi)` uses the pre-computed `GridIndex` to interpolate the cross section without repeating the binary search:

1. Compute `j = gi.bin - rxn.threshold_idx`. If $j < 0$, the energy is below threshold: return 0.
2. Read `xs_lo = xs_pool[rxn.xs_offset + j]` and `xs_hi = xs_pool[rxn.xs_offset + j + 1]`.
3. Return `xs_lo + gi.frac * (xs_hi - xs_lo)`.

This is critical for performance. A nuclide with 8 reactions would otherwise need 8 binary searches per collision; with `GridIndex` it needs 1.

10 Stochastic energy interpolation

All distribution pools (angular, Kalbach, fission energy) store tables at discrete incident energies. When a neutron has energy E between two tabulated incident energies E_k and E_{k+1}, the code must choose which table to use.

Rather than deterministically interpolating between two tables (which would require sampling from both and blending), microMC uses **stochastic interpolation**: it selects one of the two neighbouring tables probabilistically:

```
frac = (E - E_k) / (E_{k+1} - E_k)
if (uniform(rng) < frac)
    use table at E_{k+1}
else
    use table at E_k
```

Over many samples, this gives the correct interpolated distribution in expectation. It is cheaper than blending two table samples and is the standard approach in Monte Carlo transport codes.

The function `stoch_energy_interp(energy_offsets, energies, dist_id, E, rng)` implements this. It:

1. Uses `energy_offsets[dist_id]` and `energy_offsets[dist_id + 1]` to find the range of incident energies for this distribution.

2. Binary searches within that range to find the interval containing E.
3. Stochastically selects one of the two bounding entries.
4. Returns the global index j of the selected entry — which is then used with `table_offsets[j]` to find the actual CDF table.

11 Worked interpretation examples

11.1 Example 1: O16 continuum inelastic

Take O16, whose parent grid is:

```
[1e-3, 1e-1, 1e0, 1e1, 1e3, 1e5, 1e6, 1e7]
```

O16 continuum inelastic has:

```
threshold_idx = 4
xs_length = 4
```

So its xs support is parent local indices 4..7, which correspond to:

```
[1e3, 1e5, 1e6, 1e7]
```

Now suppose its Kalbach incident-energy grid is:

```
[1e3, 1e5, 1e7]
```

Notice the difference:

- the xs support is inherited from the parent grid,
- the secondary law has its own explicitly stored incident-energy grid.

They are related, but not identical.

11.2 Example 2: U235 fission

U235 parent grid is:

```
[1e-5, 1e-3, 1e-1, 1e0, 1e1, 1e3, 1e4, 1e5, 1e6]
```

U235 fission has:

```
threshold_idx = 2
xs_length = 7
```

So its xs support is parent local indices 2..8:

```
[1e-1, 1e0, 1e1, 1e3, 1e4, 1e5, 1e6]
```

But it also references:

- prompt fission outgoing-energy grid at [1e-1, 1e1, 1e5],
- $\bar{\nu}(E)$ grid at [1e-1, 1e2, 1e6],
- delayed-group metadata through `delayed_id`.

One reaction therefore touches several different pools and several different incident-energy grids.

12 Full collision pipeline

This section traces the complete data flow from "a neutron has energy E in a material" to "the neutron has a new energy and direction after collision."

12.1 Step 1: Macroscopic cross section and reaction sampling

The function `sample_reaction_and_macro_xs(mat, data, E, rng)` walks the material's nuclides and computes the total macroscopic cross section $\Sigma_t(E)$, simultaneously building a cumulative table for reaction sampling.

```
macro_xs_t = 0
for each nuclide i in material:
    nuc = data.nuclides[mat.nuclide_ids[i]]
    N_i = mat.number_densities[i]
    gi = find_grid_index(data, nuc, E) // one binary search

    for each reaction r of this nuclide:
        rxn = data.reactions[nuc.rxn_offset + r]
        sigma_r = lookup_micro_xs_at(data, rxn, gi) // reuses gi
        macro_r = N_i * sigma_r
        macro_xs_t += macro_r
        append (macro_xs_t, {nuclide_idx, rxn_idx, type}) to table
```

After the walk, sample a uniform random $\xi \in [0, \Sigma_t)$ and use `std::lower_bound` on the cumulative table to select the reaction. The result is a `ReactionChoice` containing the sampled `nuclide_idx`, `rxn_idx`, `type`, and the total Σ_t.

12.2 Step 2: Distance to collision

```
distance = -log(xi) / macro_xs_t
x += distance * Omega_x
y += distance * Omega_y
z += distance * Omega_z
```

12.3 Step 3: Target velocity sampling

Sample the target nuclide's thermal velocity from a Maxwellian at the material temperature:

$$V_i \sim \mathcal{N}\left(0, \frac{k_B T}{m_t}\right), \quad i \in \{x, y, z\}$$

Compute the relative velocity $\mathbf{g} = \mathbf{v}_n - \mathbf{V}$ and centre-of-mass velocity $\mathbf{v}_{\text{cm}} = (\mathbf{v}_n + A\,\mathbf{V})/(A+1)$.

12.4 Step 4: Dispatch to collision handler

The transport loop switches on `rxn.type` and calls the appropriate handler. Each handler uses the nuclear data in a specific way. We trace each one below.

12.5 Pipeline A: Elastic scatter

1. **Sample scattering cosine.** Call `sample_cosine(data.angular, rxn.dist_id, E, rng)`:
   (a) Stochastic energy interpolation on the angular pool's incident-energy grid to select table j.
   (b) CDF sampling on `mu[table_offsets[j] .. table_offsets[j+1]]` to get μ_{cm}.

2. **Rotate relative velocity.** Compute $\hat{\mathbf{g}}$ (unit vector along $\mathbf{g}$), then rotate it by μ_{cm} with a random azimuthal angle to get $\hat{\mathbf{g}}'$.

3. **Compute outgoing velocity.** For elastic scattering, $|\mathbf{g}'| = |\mathbf{g}|$ (relative speed is conserved). The outgoing neutron velocity in the lab frame is:

$$\mathbf{v}'_n = \mathbf{v}_{\text{cm}} + \frac{A}{A+1}\, g\, \hat{\mathbf{g}}'$$

4. **Set neutron state.** Convert $\mathbf{v}'_n$ to energy $E' = \frac{1}{2} m_n |\mathbf{v}'_n|^2$ and direction $\hat{\Omega}' = \mathbf{v}'_n / |\mathbf{v}'_n|$.

12.6 Pipeline B: Discrete inelastic scatter

Same as elastic, except:

- The outgoing relative speed changes due to the Q-value. The outgoing relative kinetic energy is $E'_{\text{rel}} = E_{\text{rel}} + Q$ (where $E_{\text{rel}} = \frac{1}{2}\mu_r\, g^2$ and μ_r is the reduced mass). If $E'_{\text{rel}} \leq 0$, the neutron is killed (insufficient energy for this level).
- The outgoing relative speed is $g' = \sqrt{2E'_{\text{rel}}/\mu_r} \neq g$.

Everything else (angular sampling from `data.angular`, CM→lab boost) is identical to elastic.

12.7 Pipeline C: Continuum inelastic scatter

1. **Sample outgoing energy and cosine.** Call `sample_kalbach_mann(data.kalbach, rxn.dist_id, E, rng)`:
   (a) Stochastic energy interpolation to select a table j.
   (b) CDF sampling on `E_out[start..end]` to get E'_{out}.
   (c) Interpolate `r` and `a` at the sampled bin.
   (d) Sample μ from the Kalbach-Mann separability formula: $f(\mu) = \frac{a}{2\sinh a}\left[\cosh(a\mu) + r\sinh(a\mu)\right]$.
2. **Compute outgoing velocity.** The outgoing speed in the CM frame is $v'_{\text{cm}} = \sqrt{2E'_{\text{out}}/m_n}$. The direction is obtained by rotating $\hat{\mathbf{g}}$ by μ.
3. **Boost to lab.** $\mathbf{v}'_n = \mathbf{v}_{\text{cm}} + v'_{\text{cm}}\, \hat{\mathbf{g}}'$.

12.8 Pipeline D: Fission

1. **Sample prompt multiplicity.** Call `sample_nu_bar(data.fission_yields, rxn.yield_id, E, rng)`:
   (a) Interpolate $\bar{\nu}(E)$ from the yield table.
   (b) Stochastically round: $\nu = \lfloor \bar{\nu} \rfloor$ with probability $\lceil \bar{\nu} \rceil - \bar{\nu}$, otherwise $\nu = \lceil \bar{\nu} \rceil$.
2. **For each prompt neutron** (ν times):
   (a) Sample outgoing energy from `sample_fission_energy(data.fission_energy, rxn.dist_id, E, rng)`.
   (b) Sample isotropic direction.
   (c) Set weight $w' = w/k_{\text{eff}}$.
   (d) Push to fission bank.
3. **Delayed neutrons** (if `rxn.delayed_id` ≥ 0):

(a) Look up `data.delayed_fission_descs[rxn.delayed_id]` to get the group block.

(b) For each delayed group $g = 0, \ldots, n_{\text{groups}} - 1$:

i. Read `data.delayed_groups[group_offset + g]`, which gives `yield_id` and `dist_id`.

ii. Sample ν_d from `sample_nu_bar(data.fission_yields, group.yield_id, E, rng)`.

iii. For each delayed neutron (ν_d times): sample E' from `sample_fission_energy(data.delayed_fissio` `group.dist_id, E, rng)`, assign isotropic direction, push to fission bank.

4. **Kill the incident neutron.**

12.9 Pipeline E: Absorption

Kill the neutron. No data access beyond setting `neutron.alive = false`.

12.10 Pipeline F: (n,2n) and (n,3n)

1. **Sample target velocity and compute CM frame** (same as elastic).

2. **Update the primary neutron:** Sample E'_{out} and μ from `sample_kalbach_mann(data.kalbach, rxn.dist_id, E, rng)`. Rotate, boost to lab, set neutron state.

3. **Create secondaries:** For $i = 1, \ldots, \text{multiplicity} - 1$: sample a fresh Kalbach result from the same distribution at the same incident energy, rotate, boost, push to secondary bank.

13 How a lookup proceeds in practice

Suppose a neutron is in a material that already identified U238 as the sampled target nuclide and U238 n2n as the sampled reaction.

Step 1: identify the nuclide block

Read `nuclides[3]`. This gives:

- U238 parent grid starts at `energy_grids[24]`,
- U238 reaction block starts at `reactions[15]`,
- there are 7 U238 reactions.

Step 2: identify the reaction descriptor

The U238 n2n channel is `reactions[20]`. It gives:

- `xs_offset = 118`,
- `xs_length = 5`,
- `threshold_idx = 4`,
- `dist_id = 7` into the Kalbach pool.

Step 3: reconstruct xs support

U238 parent grid is:

```
[1e-5, 1e-2, 1e0, 1e2, 1e3, 1e5, 1e6, 1e7, 2e7]
```

Since `threshold_idx = 4`, the reaction xs support is:

```
[1e3, 1e5, 1e6, 1e7, 2e7]
```

The corresponding values are `xs_pool[118..122]`.

Step 4: interpolate the cross section

Given the incident neutron energy, find the interval within the reconstructed support and interpolate the two neighbouring xs values. This uses the pre-computed `GridIndex` — see section 9.

Step 5: access secondary law

Use `dist_id = 7` to enter the Kalbach pool. There, use the incident-energy grid for distribution 7 to choose or interpolate the outgoing-energy law and Kalbach parameters. The stochastic energy interpolation selects one table; the CDF sampling produces E'_{out}; the Kalbach parameters r and a at the sampled bin produce μ.

That is the practical use of the structures: stepwise index chasing through well-defined pools.

14 What the design gets right

This layout is strong in several ways:

- it stores data compactly,
- it is naturally serializable (one flat binary blob),
- it avoids nested heap allocation,
- it is much closer to a GPU-friendly representation than a vector-of-vectors model,
- it clearly separates ownership (`NuclearDataHost`) from access (`NuclearData`),
- the `GridIndex` reuse pattern means N reactions share 1 binary search.

15 Common conceptual mistakes to avoid

1. Thinking every nuclide shares one xs grid. It does not. Each nuclide has its own parent grid.
2. Thinking every reaction owns its own energy grid. It does not. Reactions borrow slices of their parent nuclide's grid via `threshold_idx`.
3. Thinking the secondary incident-energy grid must match the xs support grid. It does not. Secondary distributions have independent, often coarser, incident-energy grids.
4. Thinking `dist_id` always means the same pool. It does not; the relevant pool depends on reaction type.
5. Confusing the `GridIndex` (a cached binary search result on the parent grid) with the stochastic energy interpolation index (a probabilistically selected entry on a secondary grid). They are unrelated.

6. Assuming `find_grid_index` is called per reaction. It is called per *nuclide*; all reactions of that nuclide share the result.

7. Thinking absorption and capture are separate channels. In microMC, all non-fission absorption (MT=102–117) is summed into a single `ABSORPTION` channel.

16 Final mental model

If you want one compact summary, use this:

- A `Material` says: *I am made of these nuclides at these number densities and this temperature.*
- A `NuclideDescriptor` says: *my parent grid starts here; my reactions start here.*
- A `ReactionDescriptor` says: *my xs starts here; it begins on the parent grid at this local index; my secondary law is over there.*
- A pool says: *I store many similar objects in one flat array; use offsets to reconstruct the grouping.*
- A `GridIndex` says: *the binary search has been done; here is the bin and fraction — reuse me.*

That is the whole architecture. It is not object nesting. It is a graph of slices into flat global pools.